Assignment 7: 3D Transforms and Forward Kinematics

16-384: Robot Kinematics and Dynamics

Prof. Jeff Ichnowski
Albert Xu
Henry Liao
Karthik Srinivasan

Due: 11:30 AM EST, November 5, 2026

Contents

1 Overview **3**

2 Background **4**
2.1 Parameterizations of 3D Rotations 4
2.2 3D Homogeneous Transforms 4
2.3 Forward Kinematics in 3D 5

3 Instructions **6**

4 Written Questions **7**

5 Code Questions **17**

6 Submission Checklist **20**

1 Overview

This assignment reinforces the following topics:

- Parameterizations of 3D Rotations
- 3D Homogeneous Transforms
- Forward Kinematics in 3D

2 Background

2.1 Parameterizations of 3D Rotations

Along with the 3x3 rotation matrices, there exists the Euler angles, Roll-Pitch-Yaw, axis-angle, and quaternion conventions for representing 3D rotations. The quaternion convention is used frequently in robotics because it avoids the Euler angles' issues with gimbal lock, uses only 4 numbers as compared to the 9 in rotation matrices, and is overall more numerically stable and more efficient. The conversion between the axis-angle representation and quaternion representation is shown below:

$q_x = a_x * \sin(\frac{\theta}{2})$
$q_y = a_y * \sin(\frac{\theta}{2})$
$q_z = a_z * \sin(\frac{\theta}{2})$
$q_w = \cos(\frac{\theta}{2})$
where:
the axis is normalized so: $a_x^2 + a_y^2 + a_z^2 = 1$
the quaternion is also normalized so $q_x^2 + q_y^2 + q_z^2 + q_w^2 = 1$

2.2 3D Homogeneous Transforms

As in the plane, a frame B's pose relative to a frame A is a rotation and a translation, packed into one 4×4 homogeneous transform:

$$H_B^A = \begin{bmatrix} R_B^A & p_B^A \\ 0\ 0\ 0 & 1 \end{bmatrix},$$

where R_B^A is the 3×3 rotation whose columns are B's x, y and z axes written in A, and p_B^A is B's origin written in A. A point with coordinates p^B in B has coordinates $\begin{bmatrix} p^A \\ 1 \end{bmatrix} = H_B^A \begin{bmatrix} p^B \\ 1 \end{bmatrix}$ in A. Transforms chain by multiplication, $H_C^A = H_B^A H_C^B$, and invert as $\left(H_B^A\right)^{-1} = H_A^B = \begin{bmatrix} (R_B^A)^T & -(R_B^A)^T p_B^A \\ 0\ 0\ 0 & 1 \end{bmatrix}$.

Every transform in this assignment is built from four elementary ones: a translation, and a rotation by an angle about one of the three axes,

$$Trans(x,y,z) = \begin{bmatrix} 1 & 0 & 0 & x \\ 0 & 1 & 0 & y \\ 0 & 0 & 1 & z \\ 0 & 0 & 0 & 1 \end{bmatrix} \qquad Rot_{x,\alpha} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \mathrm{c}_\alpha & -\mathrm{s}_\alpha & 0 \\ 0 & \mathrm{s}_\alpha & \mathrm{c}_\alpha & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$Rot_{y,\beta} = \begin{bmatrix} \mathrm{c}_\beta & 0 & \mathrm{s}_\beta & 0 \\ 0 & 1 & 0 & 0 \\ -\mathrm{s}_\beta & 0 & \mathrm{c}_\beta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \qquad Rot_{z,\gamma} = \begin{bmatrix} \mathrm{c}_\gamma & -\mathrm{s}_\gamma & 0 & 0 \\ \mathrm{s}_\gamma & \mathrm{c}_\gamma & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

with $\mathrm{c}_\alpha = \cos\alpha$ and $\mathrm{s}_\alpha = \sin\alpha$. Note the signs in $Rot_{y,\beta}$: about y, a positive angle turns z towards x, so the $+\mathrm{s}_\beta$ is in the top row.

Roll, pitch and yaw. An orientation can be given as three angles, roll α about x, pitch β about y and yaw γ about z, with

$$R = Rot_{z,\gamma}\, Rot_{y,\beta}\, Rot_{x,\alpha}.$$

Read right to left, that is a roll, then a pitch, then a yaw, each about the fixed axes of the frame you started in. A pose given as a position (x, y, z) and a roll-pitch-yaw (α, β, γ) is then the transform

$$H = Trans(x, y, z)\, Rot_{z,\gamma}\, Rot_{y,\beta}\, Rot_{x,\alpha}.$$

This is exactly how a URDF, the file format ROS and most simulators use to describe a robot, writes a pose: `<origin xyz="x y z" rpy="roll pitch yaw"/>`.

2.3 Forward Kinematics in 3D

Attach a frame to each link of a serial chain: frame 0 is the base, and frame i is fixed to link i, the link joint i turns.

The frame convention: every joint turns about its own z axis. Throughout this assignment, in the written questions and in the code, frame i is placed so that its z axis lies along joint i's axis of rotation, pointing the way the joint turns for a positive angle (by the right-hand rule). A joint's rotation is then always Rot_{z,q_i}, never a rotation about x or y, whichever way the joint actually points on the robot. Lining z_i up with the joint's real axis is the job of the fixed offset that comes before it. The xArm7's URDF is written the same way: every one of its seven joints has `<axis xyz="0 0 1"/>`.

Joint i sits at a fixed pose on link $i-1$, a position (x_i, y_i, z_i) and an orientation $(\alpha_i, \beta_i, \gamma_i)$ in frame $i-1$, and turns everything after it about its own z axis by its joint angle q_i. The transform across joint i is that fixed offset followed by the joint's rotation:

$$H_i^{i-1}(q_i) = \underbrace{Trans(x_i, y_i, z_i)\, Rot_{z,\gamma_i}\, Rot_{y,\beta_i}\, Rot_{x,\alpha_i}}_{\text{fixed: where joint } i \text{ sits on link } i-1}\ \underbrace{Rot_{z,q_i}}_{\text{the joint}}$$

The six numbers $[x_i, y_i, z_i, \alpha_i, \beta_i, \gamma_i]$ are joint i's row of the robot's *joint table*, and they don't change as the arm moves; only q_i does. Chaining the joints from the base out gives every frame in the base frame,

$$H_i^0 = H_{i-1}^0 H_i^{i-1}(q_i), \qquad H_0^0 = I,$$

so $H_n^0 = H_1^0 H_2^1 \cdots H_n^{n-1}$ is the end effector's pose. A frame that no joint moves, such as a tool at the end of the last link, is just its fixed offset, with no $Rot_{z,q}$ after it.

The joint table is not unique: the frames can be placed in many ways, and each placement gives different rows, but every consistent choice gives the same end effector. For the xArm7, the code uses UFACTORY's own table, from the arm's URDF.

3 Instructions

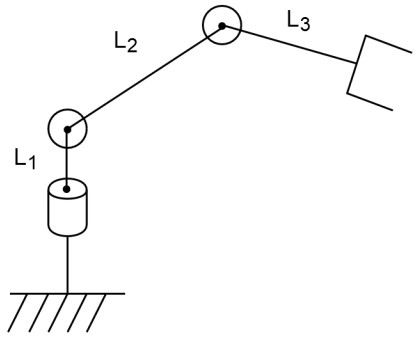
- The due date is on the title page.
- There are two separate submissions to Gradescope: your written answers as a PDF, `<andrew_id>.pdf`, and your code, as the `handin.zip` that `create_submission.py` makes. See the complete submission checklist at the end, to ensure you have everything.
- Each question (for points) is marked with a **points** heading.
- **Start early!** This homework may take a long time to complete.
- **During submission indicate the answer/page correspondence carefully when submitting on Gradescope.** If you skip a written question, just submit a blank page for it. This makes our work much easier to grade.
- If you have any questions or need clarifications, please post in Piazza or visit the TAs during the office hours.
- Unless otherwise specified, **all units are in radians, meters, and seconds**, where appropriate.

4 Written Questions

For all of the following questions, please fully compute all answers and show your work for full credit, unless otherwise specified. Submit your answers as a PDF named `<andrew_id>.pdf`. All answers must be typed, but diagrams may be hand-drawn and scanned in. However, they must be tidy and fully legible! Consider drawing them in a black or blue pen. All units are in radians and meters, where appropriate.

1) Frames and Transforms Practice

Please use the diagram of the arm below for the following question. Joint 1 (the cylinder) turns about the vertical, and joints 2 and 3 (the circles) about axes into the page. Take the arm's zero pose ($q_1 = q_2 = q_3 = 0$) to be the arm straightened out: L_1 vertical, and L_2 and L_3 in one horizontal line. Put frame 0 at joint 1 (the dot on top of the cylinder), with z_0 up, x_0 along the straightened arm, and y_0 into the page.



(1) [4 points] Sketch the arm at its zero pose, and draw frames 1, 2 and 3, one per joint, each with its z axis along its joint's axis (z_1 up, z_2 and z_3 into the page), and a tool frame 4 at the gripper. Then fill in the joint table: each frame's row $[x_i, y_i, z_i, \alpha_i, \beta_i, \gamma_i]$, its position and roll-pitch-yaw relative to the frame before it, at the zero pose.

Frame i	x_i	y_i	z_i	roll α_i	pitch β_i	yaw γ_i
1						
2						
3						
4 (tool)						

(2) [6 points] Using your table, write H_1^0, H_2^1, H_3^2 and H_4^3 (the last one is fixed: no joint turns the tool frame), and multiply them to find the gripper's position in frame 0 as a function of q_1, q_2, q_3, L_1, L_2 and L_3.

2) Parameterizations of Rotations Practice

Please convert between the different parameterizations of rotations below. **Show all of your work.**

(1) [8 points] Rotation Matrix: $R = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix}$

ZYZ Euler Angles: $\phi = ?, \theta = ?, \psi = ?$

Roll-Pitch-Yaw Angles: $Z = ?, Y = ?, X = ?$

Axis-Angle: Axis $= [?, ?, ?]$, Angle $=$?

Quaternion: $[?, ?, ?, ?]$

(2) [8 points] ZYZ Euler Angles: $\phi = \frac{\pi}{2}$, $\theta = -\frac{\pi}{2}$, $\psi = \pi$

Rotation Matrix:

$$R = \begin{bmatrix} ? & ? & ? \\ ? & ? & ? \\ ? & ? & ? \end{bmatrix}$$

Roll-Pitch-Yaw Angles: $Z = ?, Y = ?, X = ?$

Axis-Angle: Axis $= [?, ?, ?]$, Angle $= ?$

Quaternion: $[?, ?, ?, ?]$

(3) [8 points] Roll-Pitch-Yaw Angles: $X = 0$, $Y = -\frac{\pi}{2}$, $Z = 0$

Rotation Matrix:

$$R = \begin{bmatrix} ? & ? & ? \\ ? & ? & ? \\ ? & ? & ? \end{bmatrix}$$

Roll-Pitch-Yaw Angles: $Z = ?, Y = ?, X = ?$

Axis-Angle: Axis $= [?, ?, ?]$, Angle $= ?$

Quaternion: $[?, ?, ?, ?]$

(4) [8 points] Axis-Angle: Axis $= [0, 0, 1]$ Angle $= \pi$

Rotation Matrix:

$$R = \begin{bmatrix} ? & ? & ? \\ ? & ? & ? \\ ? & ? & ? \end{bmatrix}$$

Roll-Pitch-Yaw Angles: $Z = ?, Y = ?, X = ?$

Axis-Angle: Axis $= [?, ?, ?]$, Angle $= ?$

Quaternion: $[?, ?, ?, ?]$

(5) [8 points] Quaternion: $[0, 0, 0, 1]$

Rotation Matrix:

$$R = \begin{bmatrix} ? & ? & ? \\ ? & ? & ? \\ ? & ? & ? \end{bmatrix}$$

Roll-Pitch-Yaw Angles: $Z = ?, Y = ?, X = ?$

Axis-Angle: Axis $= [?, ?, ?]$, Angle $= ?$

Quaternion: $[?, ?, ?, ?]$

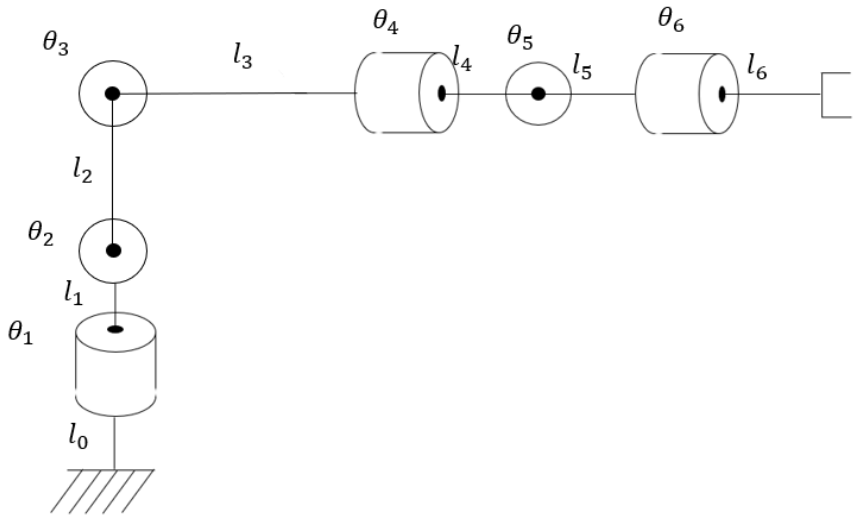


Figure 1: Side view of a 6-DOF arm, at its zero pose

3) Forward Kinematics Practice

Take the pose drawn to be the arm's zero pose (every $\theta_i = 0$). Put frame 0 on the ground at the bottom of l_0, with z_0 up, x_0 to the right and y_0 into the page. Joint 1 turns about the vertical, joints 2, 3 and 5 (the circles) about axes into the page, and joints 4 and 6 about the horizontal line of the arm.

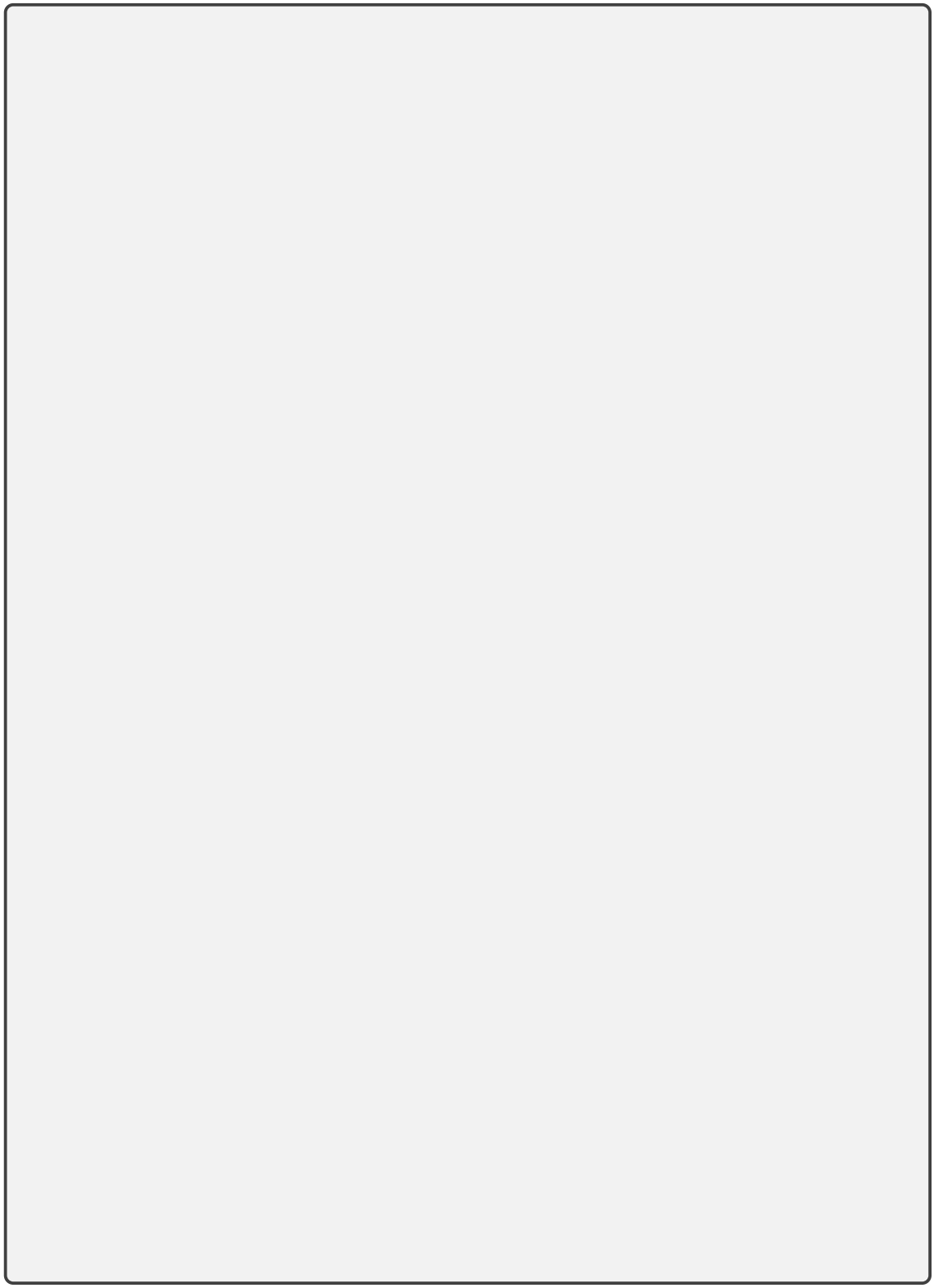
(1) [10 points] Draw frames 1 to 6, one per joint, with z_i along joint i's axis: up for joint 1, into the page ($+y_0$) for joints 2, 3 and 5, and to the right ($+x_0$) for joints 4 and 6. Add a tool frame 7 at the gripper.



(2) [20 points] Fill in the arm's joint table: each frame's row $[x_i, y_i, z_i, \alpha_i, \beta_i, \gamma_i]$, its position and roll-pitch-yaw relative to the frame before it, at the zero pose, in terms of $l_0, \ldots, l_6$. Frame 7 is fixed to link 6.

Frame i	x_i	y_i	z_i	roll α_i	pitch β_i	yaw γ_i
1						
2						
3						
4						
5						
6						
7 (tool)						

(3) [12 points] Write the transform across each joint, $H_i^{i-1}(\theta_i)$ for $i = 1, \ldots, 6$, as a 4×4 matrix, and the tool's fixed H_7^6. Then multiply them to find the gripper's position in frame 0 at the zero pose, and again with $\theta_2 = \pi/2$ and every other angle 0. Check both against the figure.



5 Code Questions

In Assignment 3 you wrote forward kinematics for a planar chain, with 3×3 homogeneous transforms built from link lengths. Here you will do the same for an arm in 3D: the xArm7 from the lab, described joint by joint as in the Background section, with 4×4 transforms. The code is in the `ex1` folder of the GitHub repository; run everything from the repository's top folder.

The graded code needs only NumPy (and Matplotlib for the sample plot). The simulator demo at the end needs the course robot library, installed from the copy in the repository, the same way as in Assignment 4:

```
cd 16384-robot-lib
conda env update -f environment.yml
conda activate 16384
cd ..
```

1) Forward Kinematics of the xArm7

Open `ex1/robot.py`. It holds the 3D transforms, a general `Robot` class for a fixed-base chain of revolute joints, as in Assignment 3, and the xArm7's joint table, from UFACTORY's URDF, as `XARM7_JOINTS`: an $n \times 6$ array with one row $[x_i, y_i, z_i, \alpha_i, \beta_i, \gamma_i]$ per joint, in metres and radians, giving joint i's position and roll-pitch-yaw in frame $i-1$.

Joint i	x_i (m)	y_i (m)	z_i (m)	roll α_i	pitch β_i	yaw γ_i
1	0	0	0.267	0	0	0
2	0	0	0	$-\pi/2$	0	0
3	0	-0.293	0	$\pi/2$	0	0
4	0.0525	0	0	$\pi/2$	0	0
5	0.0775	-0.3425	0	$\pi/2$	0	0
6	0	0	0	$\pi/2$	0	0
7	0.076	0.097	0	$-\pi/2$	0	0

Figure 2 shows the arm at its zero pose (every $q_i = 0$), with its joint axes: frame i's z axis is joint i's. The xArm7 is then

```
robot = Robot(XARM7_JOINTS)
frames = robot.fk(np.zeros(7))
```

Your code has to work for any joint table, though, not only this one. The file marks where your code goes with TODOs.

(1) [15 points] Implement the 3D transforms from the Background section, each returning a 4×4 array: `rot_x(theta)`, `rot_y(theta)` and `rot_z(theta)`, the rotations about each axis; `translation(x, y, z)`; and `transform(xyz, rpy)`, a position and a roll-pitch-yaw, built from the other four: $Trans(x, y, z)\, Rot_{z,\gamma}\, Rot_{y,\beta}\, Rot_{x,\alpha}$, in that order.

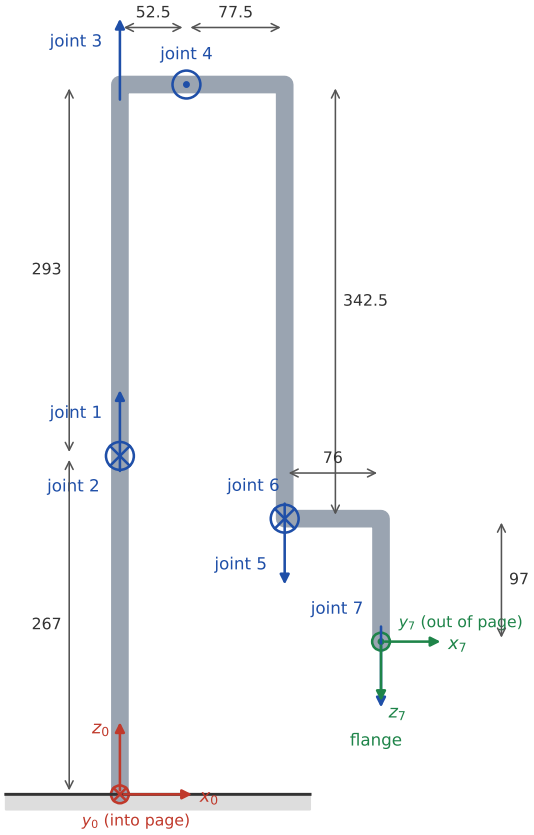


Figure 2: The xArm7 at its zero pose, side on, in mm. The view looks along $+y$, so $\otimes$ is a joint axis along $+y$ (into the page) and $\odot$ one along $-y$. Frame 0 is the base frame, and frame 7 the flange.

(2) [35 points] Implement `forward_kinematics(thetas)`, which returns every frame at once: an array `frames` of shape $(4, 4, n+1)$ with `frames[:, :, i]` $= H_i^0$, for $i = 0, 1, \ldots, n$. Frame 0 is the base frame, so `frames[:, :, 0]` is the identity; frame i is link i's, the one joint i turns; and the last frame, `frames[:, :, -1]`, is the end effector (on the xArm7, the flange). As in Assignment 3, compute the frames from the base out, each one from the frame before it. For each link, remember: its fixed offset first, from its row of the joint table (`self.joints`), then its joint's rotation by its angle from `thetas`. Following the frame convention, that rotation is always about z.

Verification and Submission

1. Run the local autograder from the repository's top folder:

   ```
   python local_autograder.py
   ```

   It runs your part (a) functions on 103 cases and your `forward_kinematics` on 98 chains (the xArm7 at 42 poses, and random chains of one to seven joints) from `ex1/inK.txt`,

compares with `ex1/outK.txt`, and says, for each failure, where the first mistake is: for part (a), which function, and for `transform`, which factor is wrong (it retries it one parameter at a time) or that they are multiplied in the wrong order; for part (b), which step of the chain is the first wrong one, and usually why: the product order, the joint's rotation before its offset, the wrong row or joint angle, or a part (a) function that is itself wrong. Add `-v` to list every failure.

2. Run `python sample_path.py` from the `ex1` folder. It plots the path your forward kinematics gives the xArm7's flange through a sample run, the arm writing RKD in the Assignment 5 lab, next to the correct path.
3. See it on the simulated xArm7:

   ```
   python local_autograder.py -p vis1
   ```

   opens the arm in your browser with a slider per joint. Your `forward_kinematics` draws all eight of its frames over the arm, and the status line checks them against the simulator's own model: your end effector against the flange, and each frame against the link it belongs to. A Matplotlib window beside it draws the same frames as a stick figure in 3D, with a cross where the simulator's flange really is.
4. The autograder on Gradescope runs the same tests, but perturbs every case's offsets, orientations and joint angles a little on each run and checks against the reference solution instead of a file. If the local check passes, you should pass on Gradescope too.
5. To submit, run `python create_submission.py` from the repository's top folder. It writes `handin.zip`; upload that to Gradescope.

6 Submission Checklist

- ☐ Upload `<andrew_id>.pdf`, your answers to the written questions, to Gradescope.
- ☐ Run `local_autograder.py` and confirm it reports no failures.
- ☐ Run `create_submission.py` and upload the resulting `handin.zip` to Gradescope.
- ☐ The code is autograded, with no limit on number of submissions within the mandated deadline.